Universität Rostock



OPERATOREN

Joern Ploennigs · AI4SC

Caligraphic Equations

Hörsaalfrage

Welche Ebenen hat die Wissenspyramide?

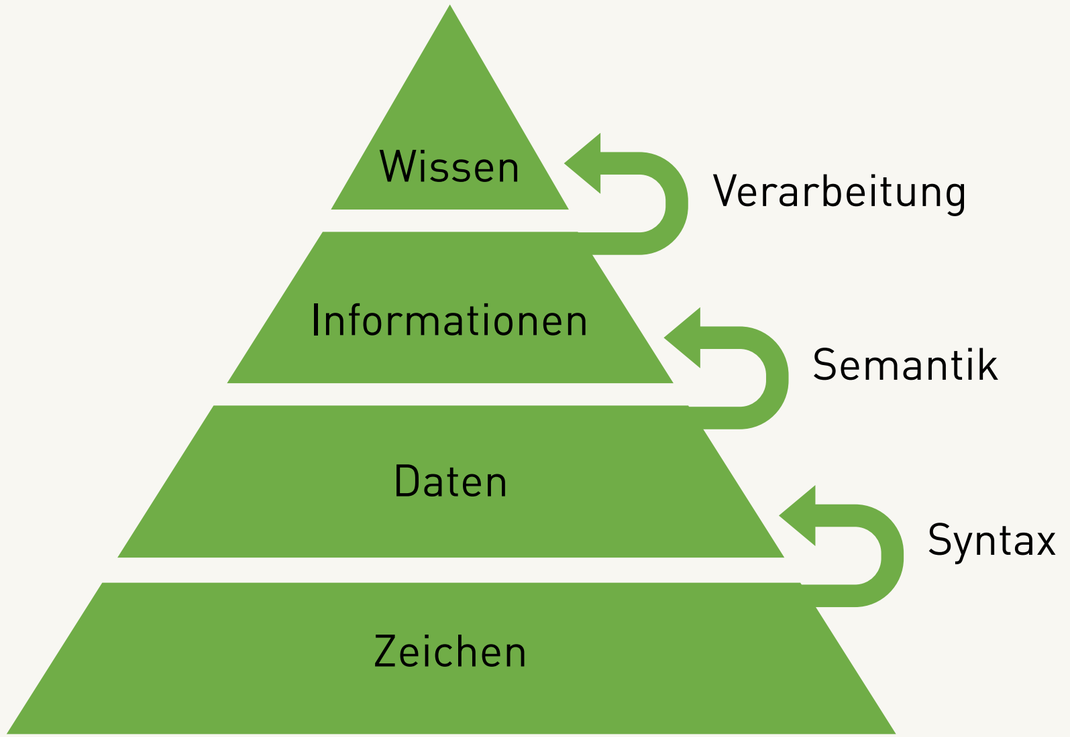

© Midjourney: Tower of Babel

Wiederholung: Wissenspyramide

Die Wissenspyramide ist ein Modell zur Darstellung der Entstehung von Wissen.

Die vier Elementtypen: *Zeichen*, *Daten*, *Informationen* und *Wissen* werden pyramidenförmig als 4 Ebenen dargestellt.

Zeichen formen die Basis und das Wissen die Spitze der Pyramide.



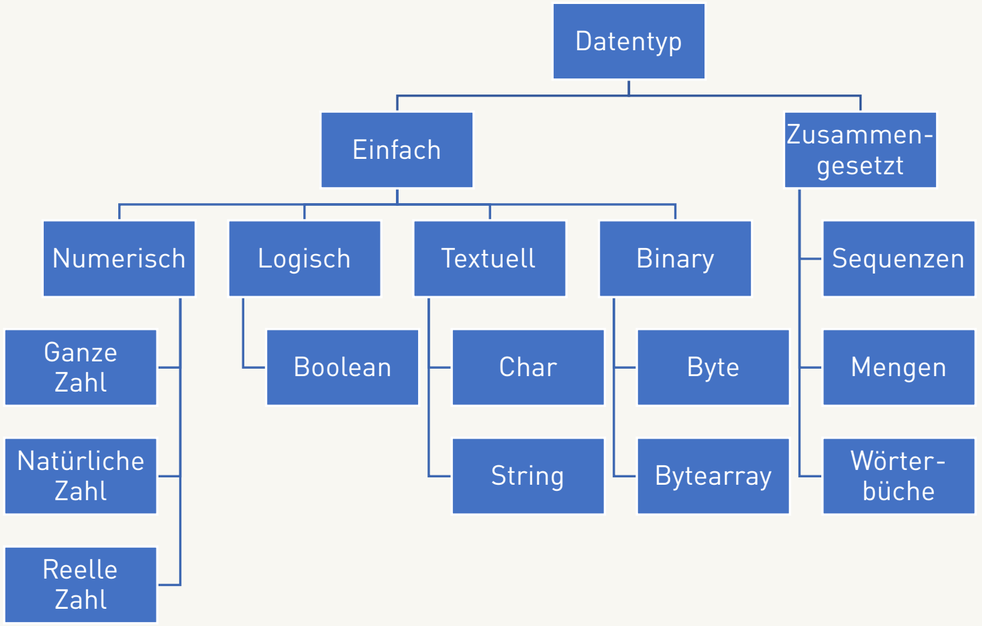
Hörsaalfrage

Welche Datentypen gibt es in Programmiersprachen?


© DALL·E 2: A chair in the shape of a avocado

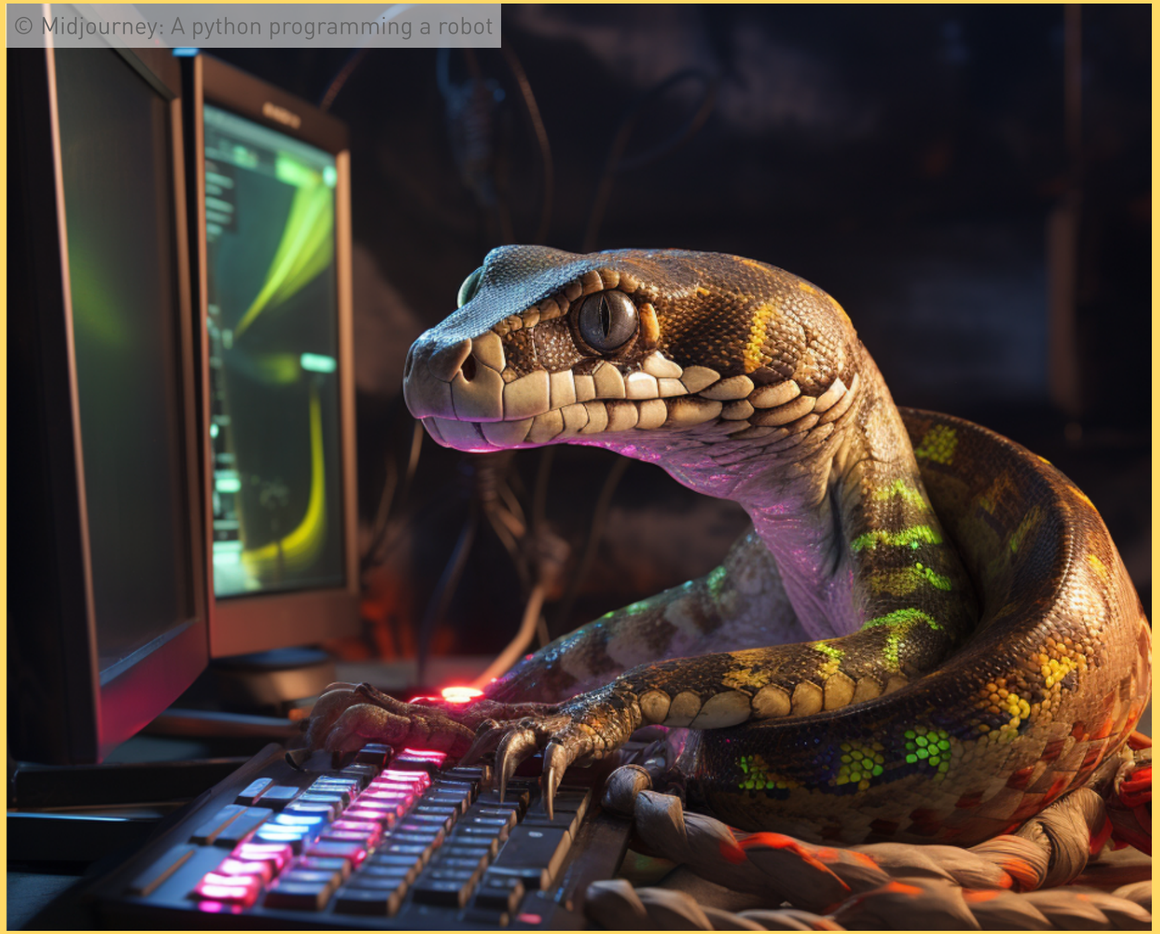
Wiederholung: Datentyp

Der Datentyp gibt an, von welcher Art die Daten sind, die mit ihm beschrieben werden (Datenvereinbarung), und welche Operationen auf diesen ausgeführt werden können.



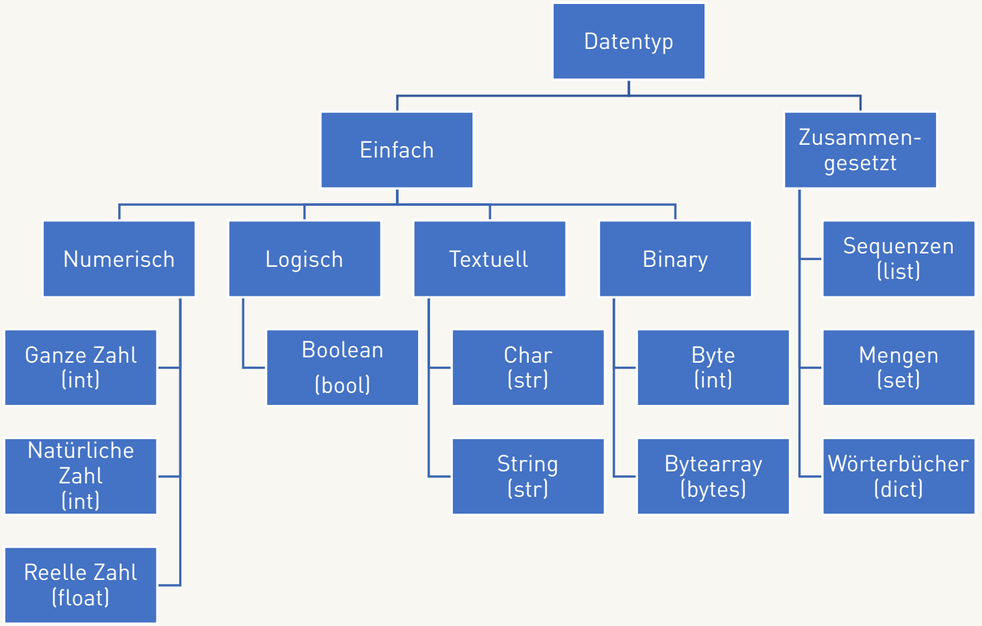
Hörsaalfrage

Welche Datentypen gibt es in Python?


© Midjourney: A python programming a robot

Wiederholung: Datentyp - Python

Python vereinfacht einige Datentypen.



Anweisung

Eine Anweisung (engl. statement) stellt eine in der Syntax einer Programmiersprache formulierte einzelne Vorschrift dar, die im Programm nacheinander auszuführen ist.

Ausdrücke

Eine besondere Form von Anweisungen sind Ausdrücke (engl. expression), die immer einen Wert zurückgeben und damit einen Datentyp haben.

Operator

Ein Operator ist ein Teil von Ausdrücken und ein Symbol oder Schlüsselwort, das dem Compiler oder Interpreter mitteilt, eine bestimmte Operation auf Werte auszuführen.

Variablenzuweisung: Ausdruck rechts, Name links

Bei einer Variablenzuweisung steht der Ausdruck rechts vom Zuweisungsoperator `=`, der Variablenname links.

```
Variablenname = Ausdruck
```

Lernziel dieser Sitzung

KI-generierte Ausdrücke lesen und nachverfolgen, statt sie von Grund auf zu schreiben.

Arithmetische Operatoren

{} Arithmetische Operatoren arbeiten mit numerischen Datentypen.

Operator	Beispiel	Ergebnis
`+`	`10 + 3`	13
`-`	`10 - 3`	7
`*`	`10 * 3`	30
`/`	`10 / 3`	3.3333...
`//`	`10 // 3`	3
`%`	`10 % 3`	1
`**`	`10 ** 3`	1000

```
x = 5
ergebnis = x ** 2 - 3 * x + 1
```

KI-generiert, erst nachverfolgen und dann ausführen

Welche Operatoren kombiniert der Ausdruck? In welcher Reihenfolge werden sie ausgewertet? Welchen Wert hat `ergebnis`?

Zuweisungsoperatoren

(×) Zuweisungsoperatoren weisen Variablen Werte zu. Python kennt kein `x++`, nur die abgekürzte Zuweisung.

Operator	Wirkung
`x = y`	`x` erhält den Wert `y`
`x += 4`	`x = x + 4`
`x *= 4`	`x = x * 4`
`x /= 4`	`x = x / 4`
`x //= 4`	`x = x // 4`
`x %= 4`	`x = x % 4`

```
a = 1 + 1
a = a + 1
a += 1
a *= 2
```

KI-generiert, erst nachverfolgen und dann ausführen

Welchen Wert hat `a` nach jeder Zeile?

Klammern verändern die Reihenfolge

```
a = 8
c1 = (6 + a) * (-2)
c2 = 6 + a * -2
```

> **KI-generiert, erst nachverfolgen und dann ausführen**
>
> Beim Lesen von KI-Code jede Änderung der Variablen mitverfolgen.

```
c1  # -28
c2  # -10
```

`c1` und `c2` enthalten dieselben Zahlen, aber die Klammern ändern die Auswertungsreihenfolge und damit das Ergebnis.

Vergleichsoperatoren

`{}` `=` weist einen Wert zu. Für den Vergleich von Werten nutzt Python `==`. Das Ergebnis ist immer `True` oder `False`.

Operator	wahr, wenn
`x == y`	`x` gleich `y`
`x != y`	`x` ungleich `y`
`x > y`	`x` größer als `y`
`x < y`	`x` kleiner als `y`
`x >= y`	`x` größer gleich `y`
`x <= y`	`x` kleiner gleich `y`

```
a = 1
a > 1
a >= 1
a == 1
a != 1
```

KI-generiert, erst nachverfolgen und dann ausführen

Welche der vier Vergleiche sind `True`?

Identitätsoperatoren

{} `x is y` ist wahr, wenn beide Variablen auf dasselbe Objekt im Speicher zeigen. `x is not y` verneint das.

- `is` prüft dieselbe Referenz.
- `==` prüft denselben Inhalt.

```
a = [1, 2, 3]
b = a
c = [1, 2, 3]
a is b
a is c
a == c
```

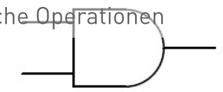
KI-generiert, erst nachverfolgen und dann ausführen

Welche der drei Ausdrücke sind `True`?

Hörsaalfrage

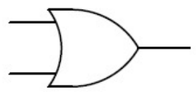
Welche Ergebnisse gibt es für welche logische Operation? (Entspricht der Aussagenlogik)

© Logische Operationen



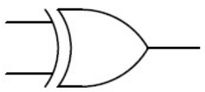
AND

A	B	Ergebnis
Falsch (0)	Falsch (0)	
Falsch (0)	Wahr (1)	
Wahr (1)	Falsch (0)	
Wahr (1)	Wahr (1)	



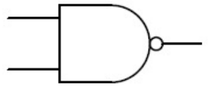
OR

A	B	Ergebnis
Falsch (0)	Falsch (0)	
Falsch (0)	Wahr (1)	
Wahr (1)	Falsch (0)	
Wahr (1)	Wahr (1)	



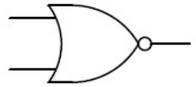
XOR

A	B	Ergebnis
Falsch (0)	Falsch (0)	
Falsch (0)	Wahr (1)	
Wahr (1)	Falsch (0)	
Wahr (1)	Wahr (1)	



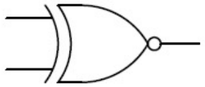
NAND

A	B	Ergebnis
Falsch (0)	Falsch (0)	
Falsch (0)	Wahr (1)	
Wahr (1)	Falsch (0)	
Wahr (1)	Wahr (1)	



NOR

A	B	Ergebnis
Falsch (0)	Falsch (0)	
Falsch (0)	Wahr (1)	
Wahr (1)	Falsch (0)	
Wahr (1)	Wahr (1)	



XNOR

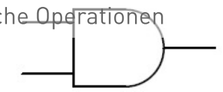
A	B	Ergebnis
Falsch (0)	Falsch (0)	
Falsch (0)	Wahr (1)	
Wahr (1)	Falsch (0)	
Wahr (1)	Wahr (1)	

N steht für `Negation`

Hörsaalfrage

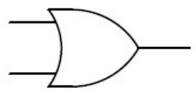
Welche Ergebnisse gibt es für welche logische Operation? (Entspricht der Aussagenlogik)

© Logische Operationen



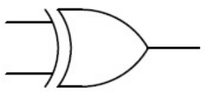
AND

A	B	Ergebnis
Falsch (0)	Falsch (0)	Falsch (0)
Falsch (0)	Wahr (1)	Falsch (0)
Wahr (1)	Falsch (0)	Falsch (0)
Wahr (1)	Wahr (1)	Wahr (1)



OR

A	B	Ergebnis
Falsch (0)	Falsch (0)	Falsch (0)
Falsch (0)	Wahr (1)	Wahr (1)
Wahr (1)	Falsch (0)	Wahr (1)
Wahr (1)	Wahr (1)	Wahr (1)



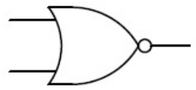
XOR

A	B	Ergebnis
Falsch (0)	Falsch (0)	Falsch (0)
Falsch (0)	Wahr (1)	Wahr (1)
Wahr (1)	Falsch (0)	Wahr (1)
Wahr (1)	Wahr (1)	Falsch (0)



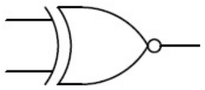
NAND

A	B	Ergebnis
Falsch (0)	Falsch (0)	Wahr (1)
Falsch (0)	Wahr (1)	Wahr (1)
Wahr (1)	Falsch (0)	Wahr (1)
Wahr (1)	Wahr (1)	Falsch (0)



NOR

A	B	Ergebnis
Falsch (0)	Falsch (0)	Wahr (1)
Falsch (0)	Wahr (1)	Falsch (0)
Wahr (1)	Falsch (0)	Falsch (0)
Wahr (1)	Wahr (1)	Falsch (0)



XNOR

A	B	Ergebnis
Falsch (0)	Falsch (0)	Wahr (1)
Falsch (0)	Wahr (1)	Falsch (0)
Wahr (1)	Falsch (0)	Falsch (0)
Wahr (1)	Wahr (1)	Wahr (1)

N steht für `Negation`

Logische Operatoren

{} Logische Operatoren kombinieren Wahrheitswerte. `P` und `Q` stehen für Ausdrücke wie `x > 10` und `x < 20`.

Operator	wahr, wenn
`P and Q`	`P` und `Q` beide wahr sind
`P or Q`	`P` oder `Q` wahr ist
`not P`	`P` falsch ist

```
alter = 14
is_teenager = (alter > 13) and (alter < 19)
```

KI-generiert, erst nachverfolgen und dann ausführen

Welchen Wert hat `is_teenager`? Welche Werte erhält man für `alter = 19`?

Operatorrangfolge: Wer bindet stärker?

Stufe	Operatoren
1 (stark)	`**`
2	`*` `/` `//` `%`
3	`+` `-`
4	`==` `!=` `<` `>` `<=` `>=` `is` `in`
5	`not`
6	`and`
7 (schwach)	`or`

```python
not x is None and x > 0.0
```

- Vergleiche (`is` , `>`) binden stärker als `not` .
- `not` bindet stärker als `and` .
- Im Zweifel Klammern setzen.

Agentischer Arbeitsablauf: Logische Ausdrücke

> **Agentic Cycle**
>
> Wenn KI `if not x is None and x > 0.0:` generiert: vor dem Ausführen nachverfolgen.

01 Vergleiche zuerst: `x is None` und `x > 0.0`

02 Dann `not` auf das Ergebnis von `x is None` anwenden

03 Zuletzt `and` verknüpft beide Teile

04 Klarere Schreibweise prüfen: `x is not None and x > 0.0`

Bitweise Operatoren

{} Bitweise Operatoren verknüpfen die Bits ganzer Zahlen. Das Gatter-Prinzip kennen Sie aus → `01c_Computerhardware`.

Operator	Wirkung
`x & y`	1, wenn beide Bits 1 sind
`x | y`	1, wenn mindestens ein Bit 1 ist
`x ^ y`	1, wenn genau ein Bit 1 ist
`~x`	Einerkomplement
`x >> n`	Bits um `n` nach rechts
`x << n`	Bits um `n` nach links

```
a = 10      # binär 1010
b = 4       # binär 0100
a & b
a | b
a << 2
a >> 2
```

KI-generiert, erst nachverfolgen und dann ausführen

Bestimmen Sie die vier Ergebnisse in Binärschreibweise, bevor Sie ausführen.

Überladen: Derselbe Operator, andere Bedeutung

{} Je nach Datentyp bedeutet derselbe Operator etwas anderes.

```
"Der " + "Ball"          # str
[1, 2, 3] + [2, 3, 4]    # list
{1, 2, 3} - {2, 3, 4}    # set
```

- `+` verbindet Strings und Listen.
- `-` bildet die Differenzmenge bei Mengen.
- `+` ist für Mengen nicht definiert.

KI-generiert, erst nachverfolgen und dann ausführen

Welche Datentypen stehen links und rechts vom Operator?

Typfehler: Häufiger Fehler in KI-Code

{} Python verknüpft nur Werte desselben Datentyps. Strings und Zahlen müssen zuerst umgewandelt werden.

```
"Der Wert ist " + 1
```

```
TypeError: can only concatenate str (not "int") to str
```

```
"Der Wert ist " + str(1)
int("1") + 1
```

Typ der Operanden prüfen

Vor dem Ausführen fragen: Welchen Datentyp haben beide Seiten? Siehe → `02b_Datentypen`.

Mitgliedsoperatoren

{} Mitgliedsoperatoren prüfen, ob ein Objekt in einer Auflistung enthalten ist.

Operator	wahr, wenn
`x in list`	`x` in `list` enthalten ist
`x not in list`	`x` nicht in `list` enthalten ist

```
4 in [1, 2, 3]
4 not in [1, 2, 3]
```

KI-generiert, erst nachverfolgen und dann ausführen

Welcher der beiden Ausdrücke ist `True`?

Übungsaufgabe: KI-Code nachverfolgen

KI hat diesen Code generiert:

```
ergebnis = not x is None and x > 0.0
```

Bestimmen Sie auf Papier den Wert von `ergebnis` für

01 `x = 10`

02 `x = None`

03 `x = "5"`

Gehen Sie wie folgt vor:

01 Operatoren und Rangfolge bestimmen

02 Jeden Teilausdruck einzeln auswerten

03 Ergebnis vorhersagen

04 Erst dann ausführen

Lösung: KI-Code nachverfolgen

Für `x = 10`:

```
x is None       # False
not False       # True
x > 0.0         # True
True and True   # True
```

Für `x = None`: `not x is None` ergibt `False`. Wegen `and` wird `x > 0.0` nicht mehr ausgewertet, das Ergebnis ist `False`.

Für `x = "5"`: `x is None` ist `False`, `not` ergibt `True`, dann scheitert `"5" > 0.0` mit einem `TypeError`.

Klarer schreiben

```
ergebnis = (x is not None) and (x > 0.0)
```

Klammern machen die Rangfolge explizit. Der Typ von `x` bleibt trotzdem zu prüfen.

Quiz zu Operatoren

Bestimmen Sie die Ergebnisse, bevor Sie ausführen:

1. `10 // 3`, `10 % 3`, `-7 // 3`
2. `0 < a < 2` mit `a = 1`
3. `[1, 2, 3] + [2, 3, 4]`

Ergebnisse:

1. `3`, `1`, `-3`
2. `True`
3. `[1, 2, 3, 2, 3, 4]`

QUESTIONS